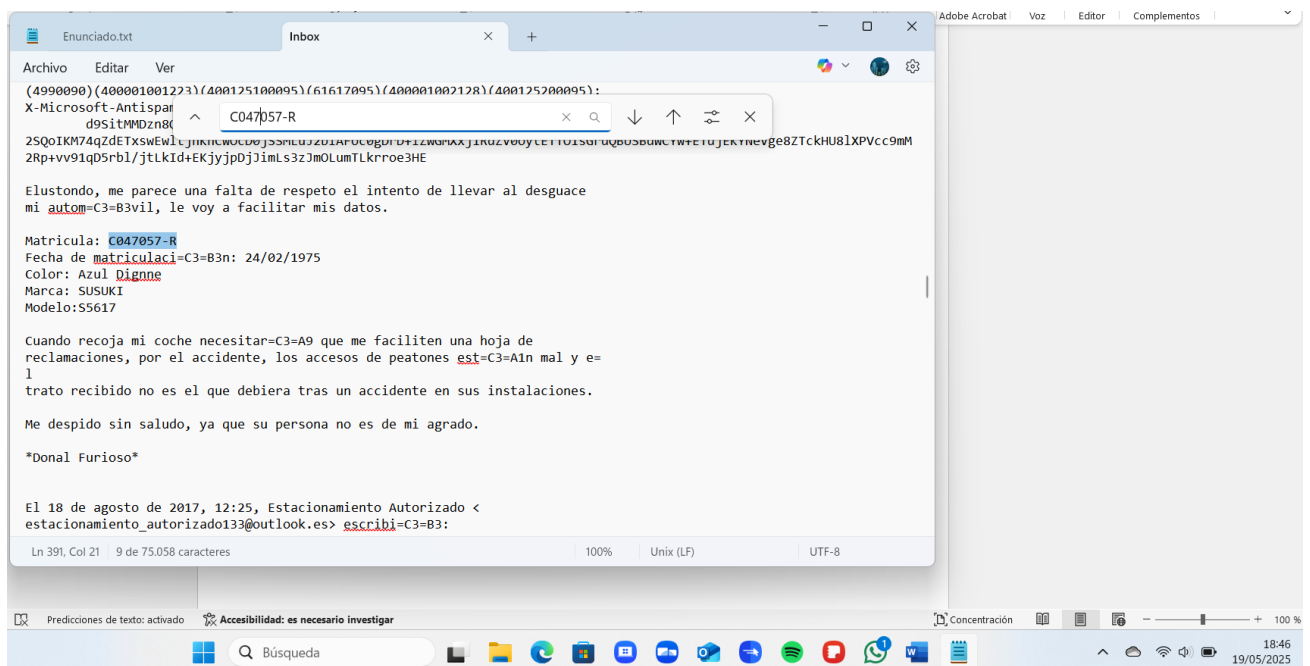


MEMORIA RETOS

1. Análisis de correos:

→**Explicación:** El reto consiste en examinar los emails de un cliente (D. Furioso) configurado en Thunderbird en modo POP3. Los correos se guardan localmente en el dispositivo. La meta es descubrir la matrícula del vehículo de D. Furioso basándose en las comunicaciones guardadas.

→**Solución:** La solución del reto consiste en la búsqueda del archivo "Inbox" que contiene los correos recibidos por el usuario en la ruta **thunderbird/ccnsqul8.default/Mail/pop-mail.outlook.com;** y una vez encontrada el archivo Inbox, dentro de él buscar el flag específico otorgado en un principio (es el número de matrícula): **C047057-R**.



2. Análisis de correos:

→**Explicación:** El reto consiste en examinar un código en Python con el fin de identificar un número concreto que, al ser introducido, genera la "flag". El código posee una gama de valores que retornan distintas secuencias de texto, aunque solo uno de estos proporciona la bandera deseada.

→**Solución:** El programa distingue entre números pares e impares dentro del rango, para números pares, tenemos los valores 88, 84 y 90; y para números impares, tenemos los valores 93, 87 y 91, cada uno devolviendo una cadena diferente.

Probamos los distintos que nos da en el código y encontramos que el único valor que corresponde con el flag dado es 87

Devuelve: **FLAG: ReAdLe\$\$tVshHsh**

→Código utilizado:

```
number = int(input("Enter a number: "))
```

```
a = 200
```

```
b = 0
```

```
a = a / 2
```

```
a = a - 7
```

```
a += 1
```

```
b += 1
```

```
b = b * 70
```

```
b = b + 12
```

```
b -= 1
```

```
m = "FLAG: "
```

```
q = "MUcH"
```

```
w = "Le$$"
```

```
e = "shHsH"
```

```
r = "ReAd"
```

```
t = "wAtcH"
```

```
y = "NoT"
```

```
u = "tV"
```

```
i = "f1Lm"
```

```
o = "B0oK"
```

```
p = "d0"
```

```
s = "pWd"
```

```
d = "C0mPuTeR"
```

```
f = "12345"
```

```
g = "54321"
```

```
h = "AbCdE"
```

```
j = "..."
```

```
if b < number < a:
```

```
    if number % 2 == 0:
```

```
        if number == 88:
```

```
            print(j + f + d + u + e)
```

```
        elif number == 84:
```

```
            print(s + r + o + j + g)
```

```
        elif number == 90:
```

```
            print(t + h + d + i + y)
```

```
        else:
```

```
            print(q + w + e + r + t)
```

```
    else:
```

```
        if number == 93:
```

```
            print(y + s + h + o + e)
```

```
        elif number == 87:
```

```
            print(m + r + w + u + e)
```

```
        elif number == 91:
```

```
            print(t + h + d + i + y)
```

```
        else:
```

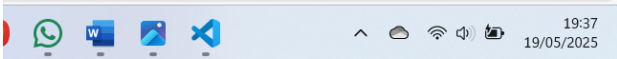
```
            print(q + w + e + r + t)
```

```
else:
```

```
    print("Nice try, but you have failed :{")
```

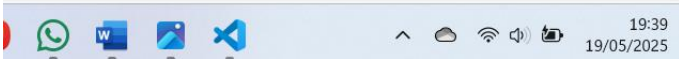
→ **Ejemplo:** Si lo probamos con 84, nos devuelve esto:

```
pWdReAdB0oK...54321  
  
[Execution complete with exit code 0]
```



Si lo probamos con 88, nos devuelve esto:

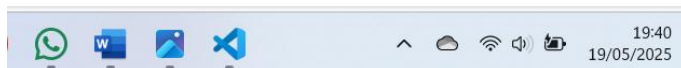
```
...12345C0mPuTeRtVshHsH  
  
[Execution complete with exit code 0]
```



Pero si lo probamos con 87, nos devuelve el flag deseado:

```
FLAG: ReAdLe$$tVshHsH
```

```
[Execution complete with exit code 0]
```

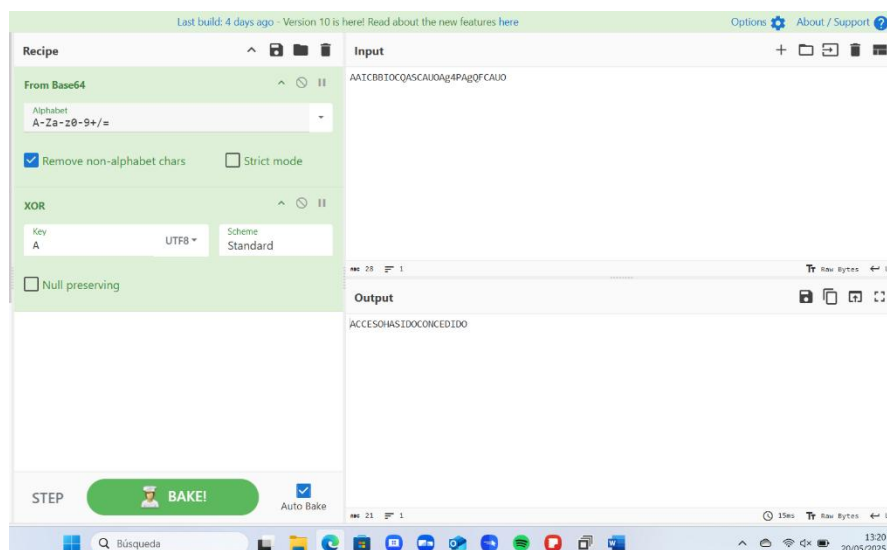


3.CrackMe:

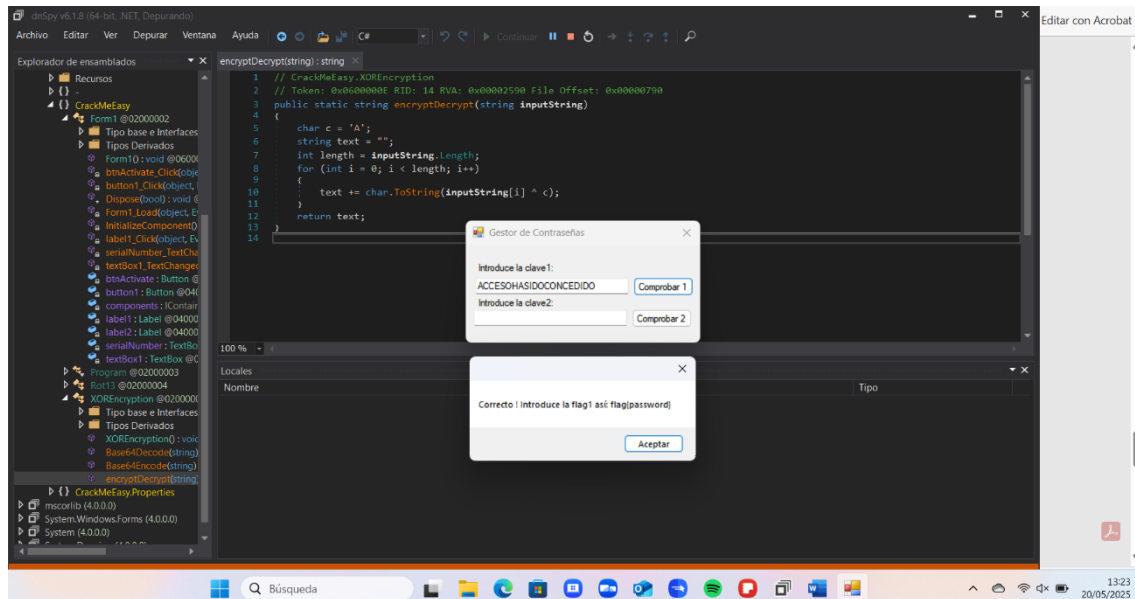
→**Explicación:** El reto consiste en examinar una aplicación ejecutable en el escritorio de un equipo informático, la aplicación posee dos entradas de texto y su propósito es obtener la primera flag (flag1) del archivo ejecutable.

→**Solución:** Buscamos en el árbol de objetos "**namespace CrackMeEasy**" dentro de Form1, luego buscamos "**btnActivate_Click**" contiene el código relevante para el flag1. Luego de esto buscamos **XOREncryption.encryptDecrypt** donde encontramos que utiliza la clave 'A'.

Abrimos CyberChef, aplicamos **From Base64**, **XOR con clave 'A'** y en input aplicamos la cadena "**AAICBBIOCQASCAUOAg4PAgQFCAUO**" y en output encontramos la palabra **ACCESOHA SIDOCONCEDIDO** (en la imagen inferior lo podemos verlo).



Tras esto ejecutamos el programa e introducimos la clave que hemos decodificado y obtenemos el resultado de la imagen inferior.

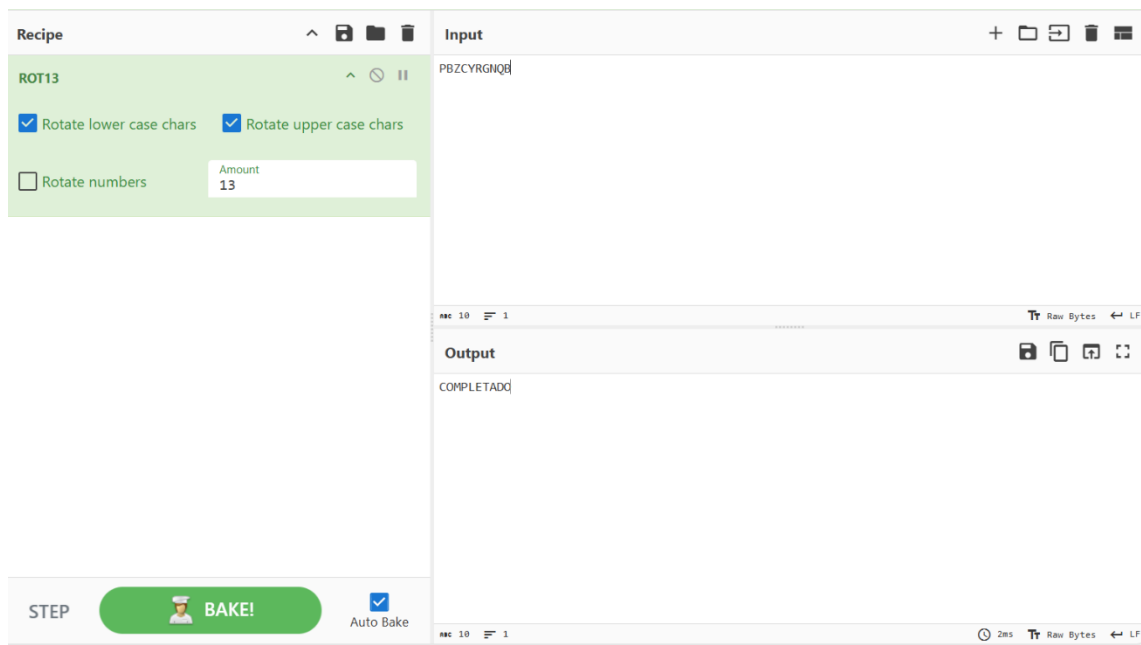


4. CrackMe 2:

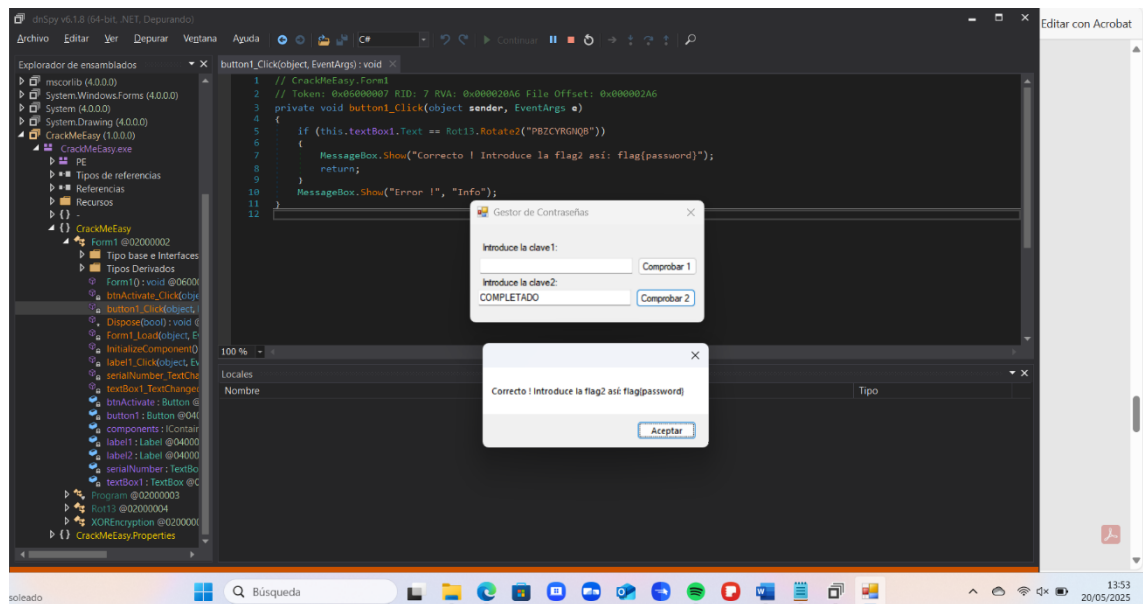
→ **Explicación:** El reto consiste en examinar una aplicación ejecutable en el escritorio de un equipo informático, la aplicación posee dos entradas de texto y su propósito es obtener la segunda flag (flag2) del archivo ejecutable.

→ **Solución:** Buscamos en el árbol de objetos "namespace CrackMeEasy" dentro de Form1, luego buscamos "button1_Click" contiene el código relevante para el flag2. Luego de esto buscamos **Rot13.Rotate2** donde encontramos se realiza un cifrado ROT13 sobre la cadena

Abrimos CyberChef, aplicamos **ROT13** y en input aplicamos la cadena "PBZCYRGNQB" y en output encontramos la palabra **COMPLETADO** (en la imagen inferior lo podemos verlo)



Tras esto ejecutamos el programa e introducimos la clave que hemos decodificado y obtenemos el resultado de la imagen inferior.



4. CrackMe 2:

→ **Explicación:** El reto consiste en examinar una aplicación ejecutable en el escritorio de un equipo informático, la aplicación posee dos entradas de texto y su propósito es obtener la segunda flag (flag2) del archivo ejecutable.

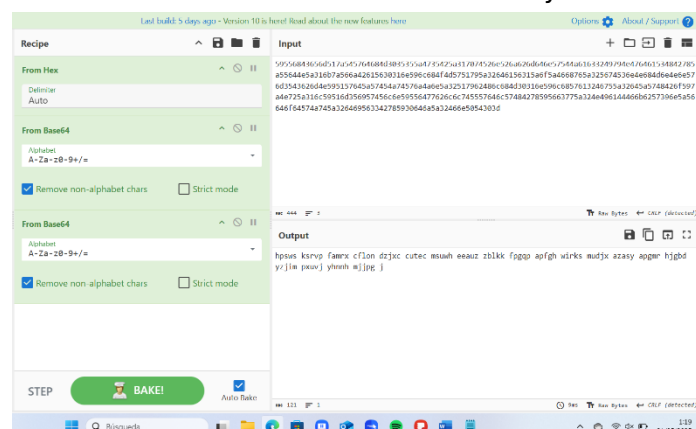
→ **Solución:** Buscamos en el árbol de objetos "namespace CrackMeEasy" dentro de Form1, luego buscamos "button1_Click" contiene el código relevante para el flag2. Luego de esto buscamos **Rot13.Rotate2** donde encontramos se realiza un cifrado ROT13 sobre la cadena

Abrimos CyberChef, aplicamos **ROT13** y en input aplicamos la cadena **"PBZCYRGNQB"** y en output encontramos la palabra **COMPLETADO** (en la imagen inferior lo podemos verlo)

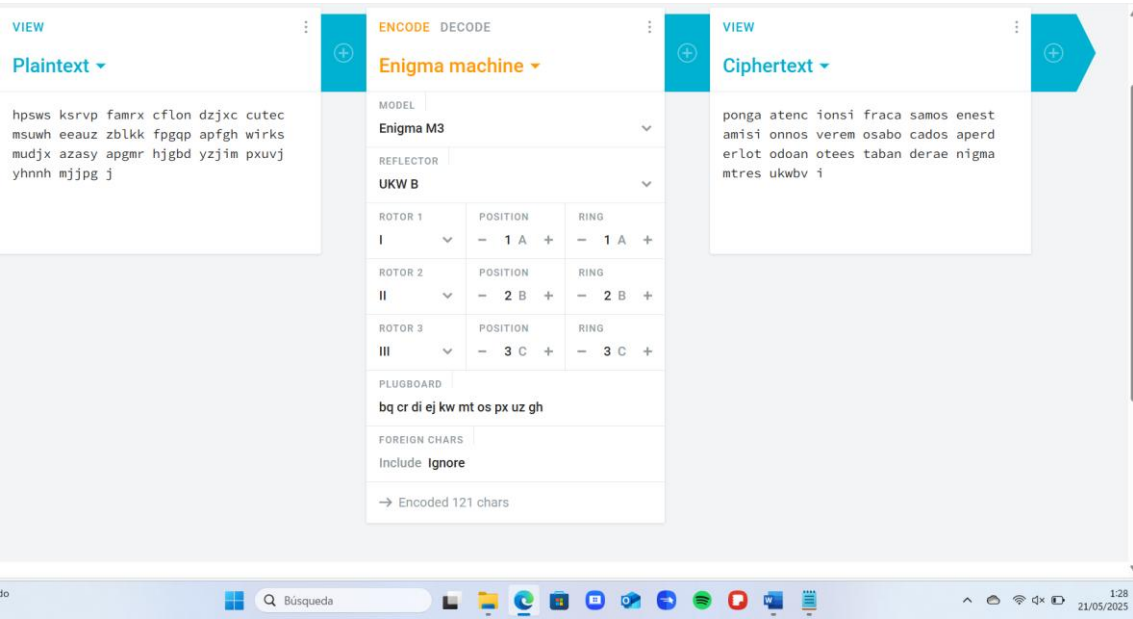
5. Enigma:

→ **Explicación:** El reto consiste en descifrar un mensaje cifrado contenido en un archivo de texto. El objetivo es utilizar las credenciales obtenidas para acceder a un ordenador y descargar un archivo que contiene información cifrada.

→ **Solución:** Primero buscamos el mensaje proporcionado en el archivo de texto que nos dan, tras esto con la herramienta CyberChef decodificamos el mensaje proporcionado que se encuentra codificado en hexadecimal y base 64.



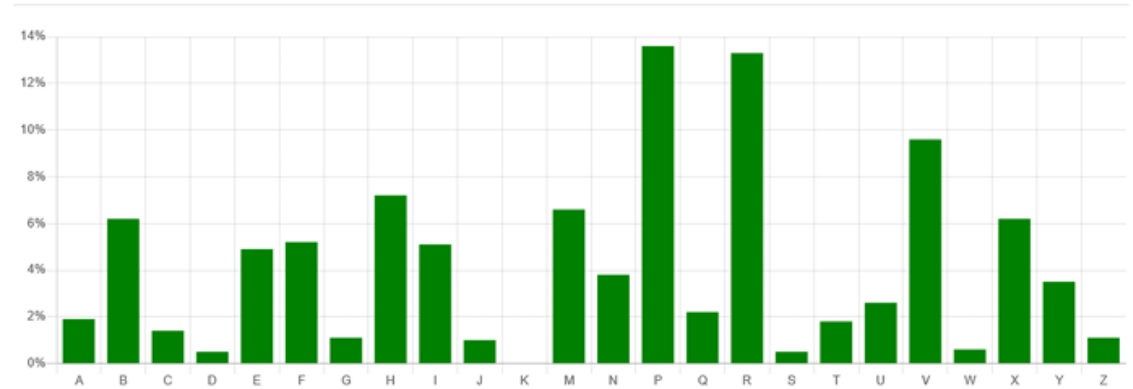
Tras esto utilizamos la herramienta cryptii que emula la máquina enigma, ponemos la configuración que nos entregaban y el mensaje decodificado hallado anteriormente y obtenemos:



6.Gana en la ruleta:

→**Explicación:** El reto consiste en aplicar conocimientos de análisis de frecuencia de caracteres para descifrar un mensaje oculto en un texto. El objetivo es encontrar una flag oculta en el texto proporcionado

→**Solución:** En este reto lo que hay que hacer es ver la frecuencia con la que aparece cada carácter en el texto que nos han otorgado, tras esto ver los caracteres más utilizados en el español e igualarlos aplicando sustitución para poder decodificar el problema.



ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz

RTNFPDJZIWOXUMVQABHYEGLKCSrtnfpdjziwoxumvqabhyeglkcs

(Los de abajo son los valores del texto original que tenemos que sustituir por los de arriba)

Y sustituyendo los valores encontrados al final del archivo
dxrj{rmrxisrmfv_xr_dbpnepmnir_pm_em_ypkyv}

Hallamos el flag: **analizando_la_frecuencia_en_un_texto**

7. Mensaje oculto:

→ **Explicación:** El reto consiste en encontrar la contraseña del archivo zip dado Mensaje, mediante la exploración de los paquetes de red y las posibles inyecciones SQL.

→ **Solución:** Para poder realizar este reto primero se tiene que explorar los paquetes de red, colocando el comando **wireshark reto.pcap** en la terminal.

Tras esto encontramos un POST que sugiere una posible inyección SQL

The image shows a web browser window with a URL bar containing a long, complex URL. The page content displays a decoded SQL injection payload. Below the browser, a Wireshark packet capture is visible, showing a POST request to a server. The packet details pane shows the request body, which is a JSON object containing a 'password' field. The packet list pane shows the request packet, and the packet bytes pane shows the raw data of the request body.

y analizando las respuestas del servidor, están son guardadas en dos ficheros distintos

```
Kali-Linux-2021.3_SS2022 [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda

~/Escritorio/mensajeoculto/02-Descargables/datos.txt - Mousepad

Fichero Editar Búsqueda Ver Documento Ayuda

1 MD5:ciencias@escuela.local;NmYzYWM5MjMzMmZiMGNiZjI2ODU1ZmYxNzFmYmI0NjI=
2 MD5:history@escuela.local;MzMDOWIxZTE5MjMxMDU2MDEyYWFkZDQxZGhkNTVjNjI=
3 MD5:informatica@escuela.local;OGI1YmF1NDI3MDdhNzJlYzdiY2hiMjZkMmYyOGZmMDQ=
4 MD5:literatura@escuela.local;OTgzZTg2MzYyYjQxZjE5NTM1ZTAzYmY1MjEwOTMwZTY=
5 MD5:matematicas@escuela.local;YmYxMmUyODk1NWY0N2VhZWZlZTQzY2hkZDZjKMDI4ZmI=
6
```

```
Kali-Linux-2021.3_SS2022 [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda

~/Escritorio/mensajeoculto/02-Descargables/datos.txt

Fichero Editar Búsqueda Ver Documento Ayuda

datos.txt

1 Exámenes.zip;1;history@escuela.local
2 Mensaje.zip;2;literatura@escuela.local
3 NOTAS.zip;3;ciencias@escuela.local
4 Temario.zip;4;informatica@escuela.local
5 Actas.zip;5;matematicas@escuela.local
6 Trimestre.zip;6;ciencias@escuela.local
7 Actividades.zip;7;matematicas@escuela.local
```

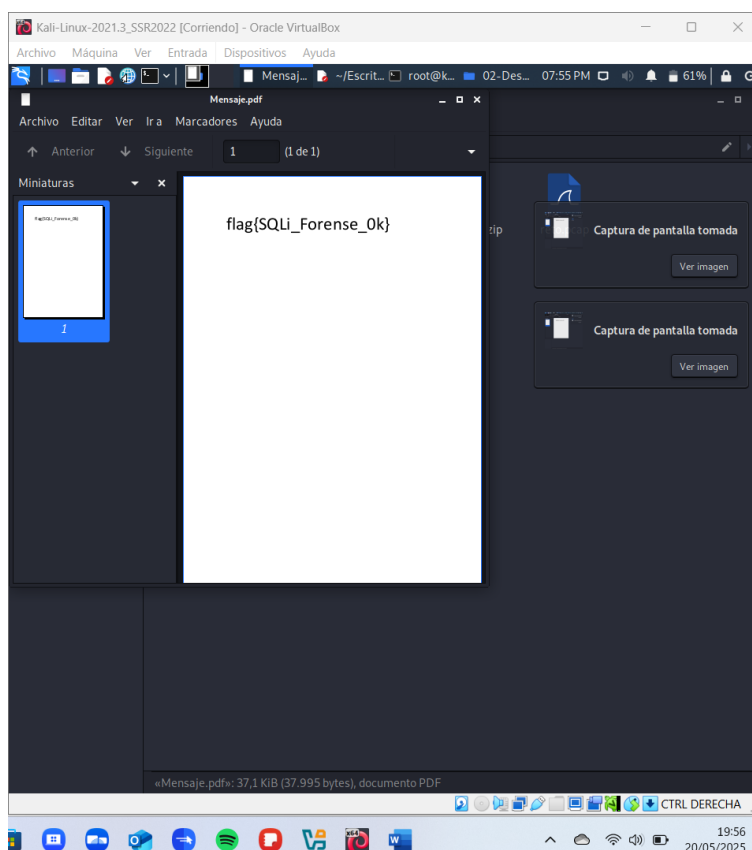
Y gracias a esto se puede relacionar el fichero con un correo electrónico y después con la contraseña en codificada en base64 y cifrada en MD5. Entonces se decodifica en base 64(imagen inferior)

```
(root@kali)-[/home/kali/Escritorio/mensajeoculto/02-Descargables]
# echo -n OTgzZTgzMzYwYjQxZjE5NTM1ZTAzYWY1MjEwOTMwZTY= | base64 -d
983e83360b41f19535e03af5210930e6
```

Y luego se descifra en MD5, dando como resultado la contraseña:

xdH9UWhNF4XGkerK

y pudiendo conseguir el pdf que se encontraba dentro del archivo **Mensaje.zip**



8.Cofre:

→**Explicación:** El reto consiste en analizar un código en Python para encontrar una flag. El programa devuelve el flag solo si el usuario introduce una cadena específica que se pueden deducir analizando el código.

→**Solución:** Básicamente lo que se ha hecho es realizar una serie de validaciones, la primera clave, a colocar debe ser un número entre el 1 y 5, la segunda clave a colocar tiene que tener el doble de longitud que la primera clave y el tercer valor tiene que cumplir con múltiples condiciones, empezar con T, debe contener exactamente 3 caracteres "a" y su longitud tiene que **ser** de 7 caracteres.

Tras esto el programa te devuelve el flag que estaban buscando que es: **"1346-ElLaberinToDePython"**

A continuación, pongo el código empleado en Python para poder resolver el reto y además unas capturas de el resultado que nos ha dado.

```
import random
```

```
from datetime import datetime
```

```
def main():
```

```
    var1 = input('Valor 1: ')
```

```
    print(var1)
```

```
    var2 = input('Valor 2: ')
```

```
    print(var2)
```

```
    if primeraClave(var1):
```

```
        if segundaClave(var1, var2):
```

```
            dameflag()
```

```
def primeraClave(clave):
```

```
    if clave in ['1', '2', '3', '4', '5']:
```

```
        print(f"Has elegido el numero {clave}")
```

```
        return True
```

```
    else:
```

```
        print('No has pasado el primer control.')
```

```

    return False

def segundaClave(clave, clave2):
    if len(clave2) != (2 * int(clave)):
        print("No has pasado el segundo control")
        return False
    else:
        print('Segundo control pasado')
        return True

def dameflag():
    sleep_time = random.randrange(1, 5)
    word = "jaja"
    while len(word) != 7:
        print("Buu!", word)
        word = input("Introduce la password: ")
        print(word)

    if word.startswith('T'):
        if word.count(chr(97)) == 6:
            res = 'flag:000000000000'
            word.endswith('d')
            res = 'no ' + res + 'Python'
            print(res)
        elif word.count(chr(97)) == 3:
            res = 'flag: 43 6c 61 72 6f 51 75 65 53 69 47 75 61 70 69'
            res = "
            if word.endswith('d'):

```

```

        print("Ok, gracias")

        res = '1346-ElLaberintoDePython'

        print("The flag is: " + res)
    else:

        res = 'flag:000000000000'

        res = 'no ' + res + 'Python'

        now = datetime.now()

        mm = str(now.month)

        dd = str(now.day)

        yyyy = str(now.year)

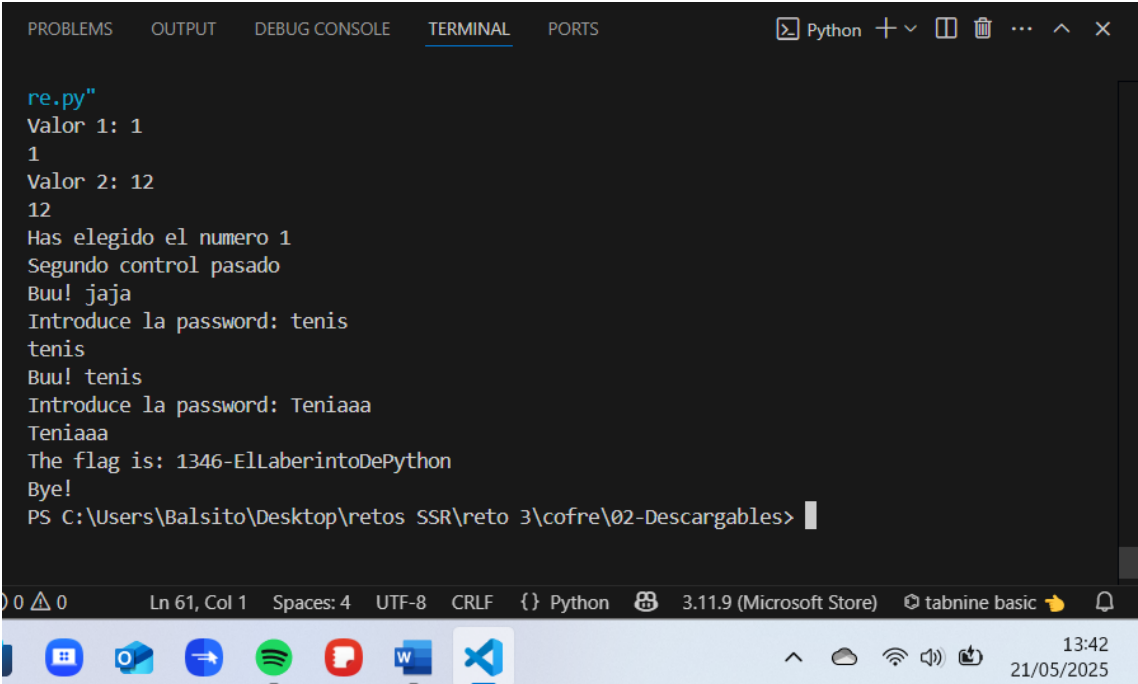
        print(dd + '/' + mm + '/' + yyyy)

    print("Bye!")

if __name__ == "__main__":

    main()

```



The screenshot shows a VS Code terminal window with the following output:

```

re.py"
Valor 1: 1
1
Valor 2: 12
12
Has elegido el numero 1
Segundo control pasado
Buu! jaja
Introduce la password: tenis
tenis
Buu! tenis
Introduce la password: Teniaaa
Teniaaa
The flag is: 1346-ElLaberintoDePython
Bye!
PS C:\Users\Balsito\Desktop\retos SSR\reto 3\cofre\02-Descargables>

```

The terminal window has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (active), and PORTS. The status bar at the bottom shows the file path, line/col info, encoding, and the Python version (3.11.9).

9. Rompecabezas:

→ **Explicación:** El reto consiste en resolver un rompecabezas y encontrar el mensaje oculto creado por profesores de diversas materias para sus alumnos.

→ **Solución:** Primero que todo nos dan un documento de texto donde nos dan distintos datos en el primero nos hablan sobre el modelo y nos dice que son una suma de dos imágenes y las buscamos en internet, la primera es sobre una máquina enigma y la segunda es Bryggen en Noruega, entonces si buscamos información online sobre los distintos tipos de máquinas enigmas que existen, encontramos una variante que es la variante Noruega. Entonces ya tendríamos el modelo.

Modelo: Enigma I Norenigma

Si traducimos las palabras que nos dan en el traductor (Nos pone por defecto que es alemán)

Posición del rodillo: V II III

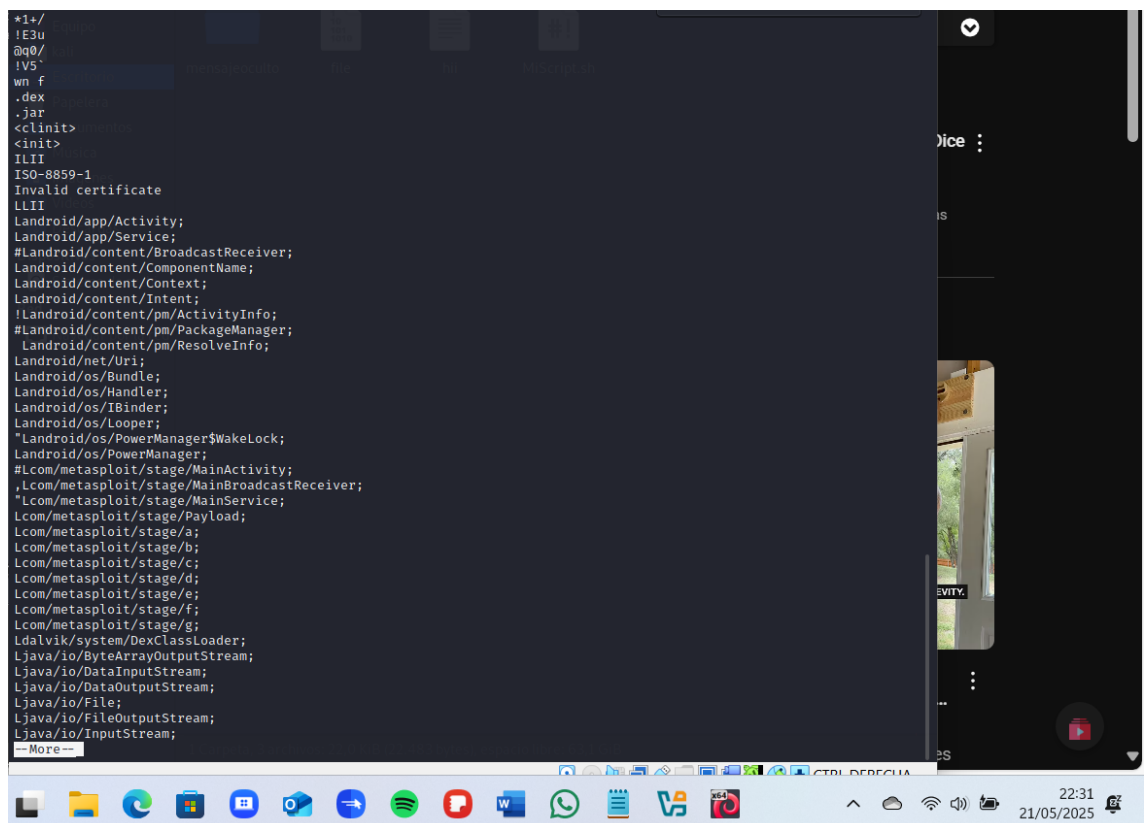
Posición del anillo: 13 17 04

Conexiones de enchufe: gu cr di ej kw mt ps ox qz bh

Hemos de entender que la última línea es el mensaje cifrado que tenemos que descifrar **niwyy lufqj yeegx hpbas wnkk ygtwwd rxmhs cczuh trtsu lssje idaxh axzj**

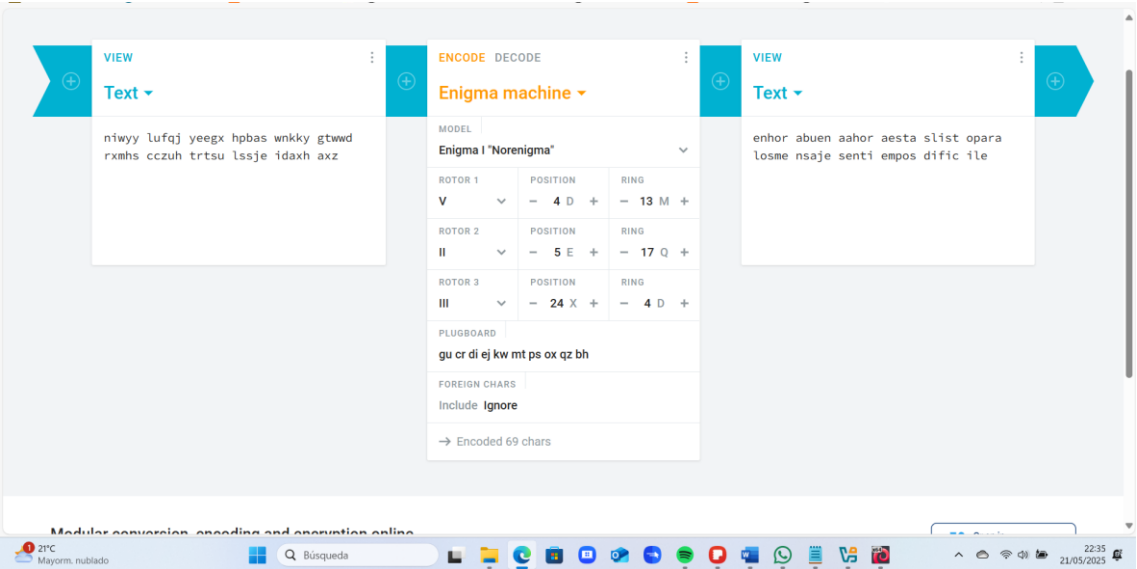
Pero los valores de posición no lo tenemos, pero nos dan la pista que es la extensión de archivo “file”

Tras esto buscamos en el archivo, pero nos damos cuenta de que el magic header del archivo ha sido borrado por lo que buscamos los strings en el archivo con **strings file | more**



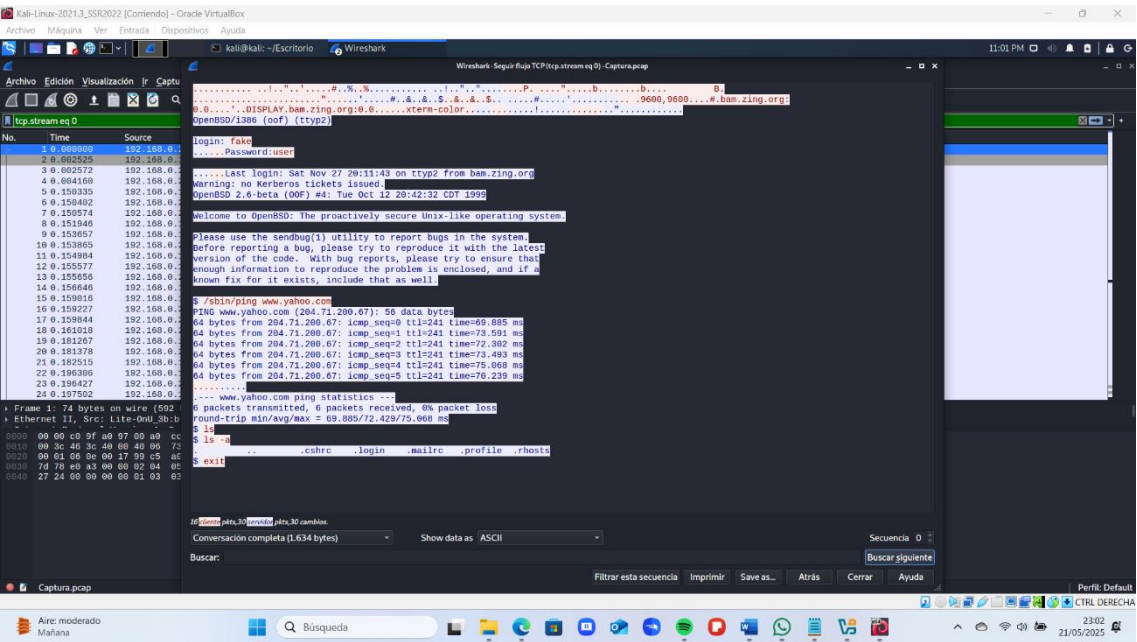
```
*1+/  
!E3u  
@q0/  
!V5  
wn f  
.dex  
.jar  
<clinit>  
<init>  
IIII  
ISO-8859-1  
Invalid certificate  
IIII  
Landroid/app/Activity;  
Landroid/app/Service;  
#Landroid/content/BroadcastReceiver;  
Landroid/content/ComponentName;  
Landroid/content/Context;  
Landroid/content/Intent;  
!Landroid/content/pm/ActivityInfo;  
#Landroid/content/pm/PackageManager;  
Landroid/content/pm/ResolveInfo;  
Landroid/net/Uri;  
Landroid/os/Bundle;  
Landroid/os/Handler;  
Landroid/os/IBinder;  
Landroid/os/Looper;  
*Landroid/os/PowerManager$WakeLock;  
Landroid/os/PowerManager;  
#Lcom/metasploit/stage/MainActivity;  
Lcom/metasploit/stage/MainBroadcastReceiver;  
*Lcom/metasploit/stage/MainService;  
Lcom/metasploit/stage/Payload;  
Lcom/metasploit/stage/a;  
Lcom/metasploit/stage/b;  
Lcom/metasploit/stage/c;  
Lcom/metasploit/stage/d;  
Lcom/metasploit/stage/e;  
Lcom/metasploit/stage/f;  
Lcom/metasploit/stage/g;  
Ldalvik/system/DexClassLoader;  
Ljava/io/ByteArrayOutputStream;  
Ljava/io/DataInputStream;  
Ljava/io/DataOutputStream;  
Ljava/io/File;  
Ljava/io/FileOutputStream;  
Ljava/io/InputStream;  
--More--
```

Encontramos dos tipos de archivos .dex y .jar y probamos esta tipo de combinación (.dex) en la máquina enigma obteniendo el mensaje descifrado



10.Tramas en la red:

- **Explicación:** Debemos analizar una captura de tráfico de red para encontrar las credenciales utilizadas en una conexión remota a través de Telnet.
- **Solución:** Debemos analizar el archivo “Captura.pcap” e identificar si existe un protocolo conocido de conexión remota en la captura, al haber tan pocos paquetes podemos ver que la conexión utilizada es Telnet y siguiendo el flujo TCP de la red podemos ver que el LOGIN es fake y su PASSWORD es user, como se puede apreciar en la imagen.

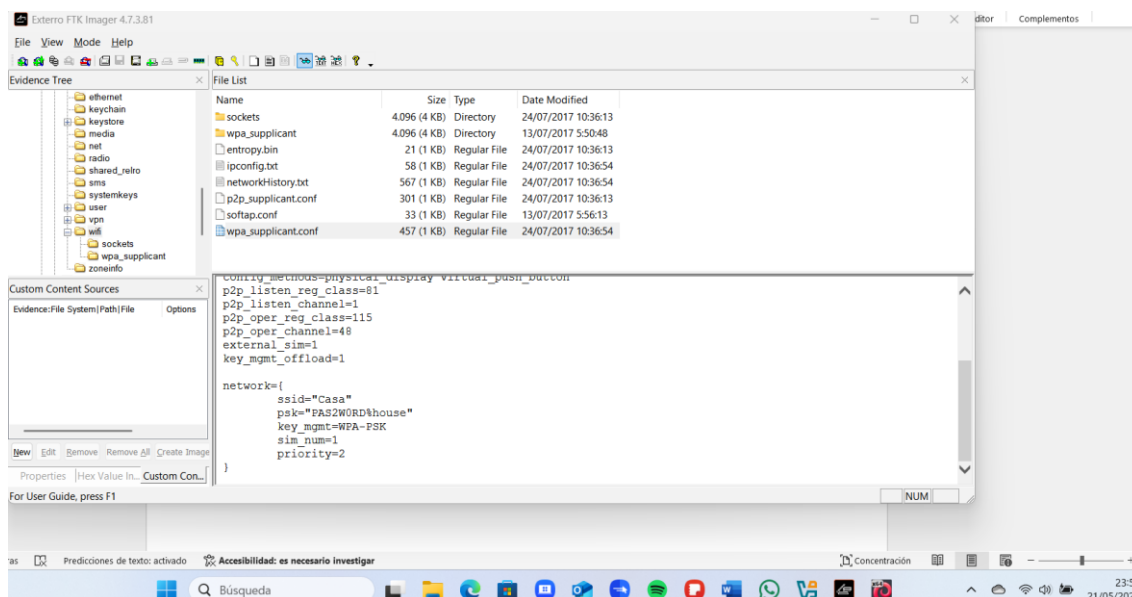


11.Dame Tu WIFI Primo:

→ **Explicación:** El reto consiste en adquirir la contraseña del WiFi de la red "Casa" mediante una imagen RAW del sistema de archivos de un teléfono móvil Android. La meta es explorar la estructura de la imagen con el fin de hallar la contraseña del WiFi.

→ **Solución:** Lo primero que tenemos que hacer es abrir el programa FTK Imager, ir a la parte superior dar a File, luego a Add Evidence Item, elegir la opción Image File y tras esto, colocar la imagen que nos dan al bajar los archivos del reto.

Tras esto buscamos la ruta /misc/wifi y encontramos el fichero **wpa_supplicant.conf**



Y en esta encontramos la contraseña **PAS2WORD%house** que se puede visualizar en la imagen.

11.Dame Tu WIFI Primo:

→ **Explicación:** El reto consiste en que hay que examinar el contenido del disco y la memoria cifrada para recuperar la información del profesor de tecnología. El disco se encuentra encriptado y se quiere hallar la clave para revelarlo.

→ **Solución:** Lo primero que hay que hacer es descomprimir el archivo DiscoCifrado y dentro encontramos una carpeta que contiene dos archivos el archivo disk_image.vmdk (fichero que representa un disco duro de una máquina virtual) y un archivo memdump.raw

Intentamos montar la imagen del disco para ver el contenido con guestmount pero no nos deja debido a que necesitamos una contraseña que no tenemos

12.Evidencias:

→ **Explicación:** El reto consiste en examinar un dump de memoria de un pendrive con el fin de recuperar datos escondidos. La meta es localizar un archivo comprimido en

formato ZIP con contraseña, adquirir dicha contraseña y posteriormente diseñar adecuadamente las cabeceras del archivo que lo contiene.

→**Solución:** Lo primero ejecutamos una serie de comandos como encontramos en la imagen inferior

```
kali@kali: ~/Desktop/evidencias/02-Descargables
File Actions Edit View Help
total 65537
-rw-rw-rw- 1 kali kali 67108864 May 23 12:10 cap.bin
drwxr-xr-x 4 root root 512 Dec 31 1969 mounted

(kali@kali)~[~/Desktop/evidencias/02-Descargables]
$ ls -la mounted/a.zip
-rwxr-xr-x 1 root root 6968 Nov 15 2017 mounted/a.zip

(kali@kali)~[~/Desktop/evidencias/02-Descargables]
$ ls -la mounted/*
-rwxr-xr-x 1 root root 6968 Nov 15 2017 mounted/a.zip

mounted/Docs:
total 16
drwxr-xr-x 2 root root 512 Nov 15 2017 .
drwxr-xr-x 4 root root 512 Dec 31 1969 ..
-rwxr-xr-x 1 root root 7309 Nov 15 2017 important.pdf
-rwxr-xr-x 1 root root 6963 Nov 15 2017 manual.pdf

mounted/Photos:
total 1234
drwxr-xr-x 2 root root 512 Nov 15 2017 .
drwxr-xr-x 4 root root 512 Dec 31 1969 ..
-rwxr-xr-x 1 root root 775682 Nov 15 2017 cat.jpg
-rwxr-xr-x 1 root root 485609 Nov 15 2017 kitty.jpg

(kali@kali)~[~/Desktop/evidencias/02-Descargables]
$
```

Ahora realizamos una data carving con la herramienta foremost, para esto primero hay que instalarla en Kali con el comando `sudo apt-get install foremost`.

Tras esto colocamos los comandos `foremost cap.bin` y `cd output/`.

Aplicamos `ls -la *`, entonces nos sale estos archivos de la imagen inferior

```
root@kali: /home/kali/Desktop/evidencias/02-Descargables/output
File Actions Edit View Help
$ ls -la *
-rw-r--r-- 1 root root 1066 May 23 12:42 audit.txt

jpg:
total 2356
drwxr-xr-- 2 root root 4096 May 23 12:42 .
drwxr-xr-- 5 root root 4096 May 23 12:42 ..
-rw-r--r-- 1 root root 775682 May 23 12:42 00114210.jpg
-rw-r--r-- 1 root root 485609 May 23 12:42 00115726.jpg
-rw-r--r-- 1 root root 991125 May 23 12:42 00116675.jpg
-rw-r--r-- 1 root root 145057 May 23 12:42 00118611.jpg

pdf:
total 44
drwxr-xr-- 2 root root 4096 May 23 12:42 .
drwxr-xr-- 5 root root 4096 May 23 12:42 ..
-rw-r--r-- 1 root root 17484 May 23 12:42 00114145.pdf
-rw-r--r-- 1 root root 6963 May 23 12:42 00114180.pdf
-rw-r--r-- 1 root root 7309 May 23 12:42 00114194.pdf

zip:
total 24
drwxr-xr-- 2 root root 4096 May 23 12:42 .
drwxr-xr-- 5 root root 4096 May 23 12:42 ..
-rw-r--r-- 1 root root 6968 May 23 12:42 00118875.zip
-rw-r--r-- 1 root root 5485 May 23 12:42 00118889.zip

(root@kali)~[~/Desktop/evidencias/02-Descargables/output]
```

Tras esto ponemos el comando # crunch 5 5 -t ,%^@ -o dict.txt, para generar un diccionario con contraseñas, # fcrackzip -D -p dict.txt -u 00118889.zip, obteniendo la contraseña que encontramos en la parte inferior.

```

root@kali: /home/kali/Desktop/evidencias/02-Descargables/output/zip
File Actions Edit View Help
drwxr-xr-x 4 root root      512 Dec 31 1969 mounted
drwxr-xr-- 5 root root     4096 May 23 13:09 output

(root@kali)-[/home/kali/Desktop/evidencias/02-Descargables]
# cd output

(root@kali)-[/home/.../Desktop/evidencias/02-Descargables/output]
# cd zip

(root@kali)-[/home/.../evidencias/02-Descargables/output/zip]
# crunch 5 5 -t ,%^@ -o dict.txt
Crunch will now generate the following amount of data: 13384800 bytes
12 MB
0 GB
0 TB
0 PB
Crunch will now generate the following number of lines: 2230800

crunch: 100% completed generating output

(root@kali)-[/home/.../evidencias/02-Descargables/output/zip]
# fcrackzip -D -p dict.txt -u 00118889.zip

PASSWORD FOUND!!!!: pw = E94$z

(root@kali)-[/home/.../evidencias/02-Descargables/output/zip]
#

```

Descomprimos el fichero con la contraseña obtenida anteriormente

```

(root@kali)-[/home/.../evidencias/02-Descargables/output/zip]
# unzip 00118889.zip
Archive: 00118889.zip
[00118889.zip] File password:
replace File? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
inflating: File

(root@kali)-[/home/.../evidencias/02-Descargables/output/zip]
# ls
00118875.zip 00118889.zip dict.txt File

(root@kali)-[/home/.../evidencias/02-Descargables/output/zip]
# file File
File: PDF document, version 1.4

```

Parece ser un archivo pdf, pero si abrimos un editor hexadecimal (hexeditor) y revisamos el contenido debemos ver en la parte superior que pone PNG, pues modificamos la cabecera del archivo para que tenga la cabecera de un PNG, como podemos ver en la imagen de abajo.

```

root@kali: /home/kali/Desktop/evidencias/02-Descargables/output/zip
File Actions Edit View Help

File: File
ASCII 0FFs: 0-00000000 / 0-00000000 (100)
00000000 09 50 4E 47 0D 0A 1A 0A 00 00 00 0D 49 48 44 52 PNG.....IHDR
00000010 00 00 03 42 00 00 01 79 08 02 00 00 00 3D 1E 0D ..B...y.....=...
00000020 64 00 00 00 01 73 52 47 42 00 AE CE 1C E9 00 00 ....sRGB.....
00000030 00 04 67 41 4D 41 00 00 B1 8F 0B FC 61 05 00 00 .gAMA.....a...
00000040 00 09 70 48 59 73 00 00 0E C3 00 00 0E C3 01 C7 .pHYs.....
00000050 6F A8 64 00 00 19 F6 49 44 41 54 78 5E ED DB ED .d....IDATx^...
00000060 81 1B B9 B1 05 50 C7 B5 01 29 9E 8D 66 93 71 30 ...P... )..f.q0
00000070 B6 46 9A D5 7C 35 81 0B 34 D0 64 71 CE F9 ED 01 F..|5..4.dq...
00000080 0B 85 02 FA 3E F9 F9 3F FF 03 00 A0 20 31 0E 00 ...>..?....1..
00000090 A0 24 31 0E 00 A0 24 31 0E 00 A0 24 31 0E 00 A0 $1...$1...$1...
000000A0 24 31 0E 00 A0 24 31 0E 00 A0 24 31 0E 00 A0 24 1...$1...$1...$
000000B0 31 0E 00 A0 24 31 0E 00 A0 24 31 0E 00 A0 24 31 ...$1...$1...$1
000000C0 0E 00 A0 24 31 0E 00 A0 24 31 0E 00 A0 24 31 0E ..$1...$1...$1
000000D0 00 A0 24 31 0E 00 A0 24 31 0E 00 A0 24 31 0E 00 $1...$1...$1

```

En las 2 imágenes de abajo podemos apreciar como el archivo a cambiado a PNG y que si le damos al comando open File, nos muestra el contenido de dicho archivo.

```
(root@kali)-[/home/.../evidencias/02-Descargables/output/zip]
# hexeditor

(root@kali)-[/home/.../evidencias/02-Descargables/output/zip]
# file File
File: PNG image data, 834 x 377, 8-bit/color RGB, non-interlaced

(root@kali)-[/home/.../evidencias/02-Descargables/output/zip]
# open file
```

